



Lista de exercícios - 03 – Teoria dos Grafos

Prof. Gustavo Pinheiro

Abril de 2026

1 Grafos Acíclicos Direcionados (DAGs)

1.1. **Grafos Acíclicos Direcionados (DAGs): Conceito Fundamental:** Um DAG é um tipo especial de grafo direcionado que não contém ciclos direcionados.

- Enuncie a definição formal de um DAG e cite exemplos práticos de estruturas que podem ser modeladas como DAGs.
- Explique por que a ausência de ciclos é crucial para aplicações como ordenação de tarefas e resolução de dependências.

- Um **DAG** é um grafo direcionado $G = (V, E)$ no qual não existe caminho direcionado que inicia e termina no mesmo vértice (ou seja, nenhum ciclo direcionado). Exemplos: ordenação de disciplinas em um currículo (Matemática $\rightarrow$ Cálculo $\rightarrow$ Eq. Diferenciais); compilação de arquivos (código.c $\rightarrow$ lib.o $\rightarrow$ executavel); construção de um projeto com dependências de tarefas.
- Sem ciclos, é possível definir uma ordem linear entre as tarefas/atividades. Se houvesse um ciclo ($A \rightarrow B \rightarrow C \rightarrow A$), criaria uma dependência circular impossível de satisfazer (A precisa de B, B precisa de C, C precisa de A). DAGs garantem que sempre existe uma **ordenação válida**.

1.2. **Equivalência: DAG $\Leftrightarrow$ Ordenação Topológica Existe:** Uma caracterização fundamental de DAGs.

- Prove informalmente por que “um grafo direcionado é DAG se e somente se admite uma ordenação topológica”.
- Se um grafo **não** é um DAG (contém ciclo), por que é impossível encontrar uma ordenação topológica?

- ($\Rightarrow$) Se é DAG, faz-se uma DFS e ordena-se os vértices em ordem decrescente de tempo final (*finishing time*). Isso garante que para toda aresta $u \rightarrow v$, u aparece antes de v na ordenação. ($\Leftarrow$) Se existe ordenação topológica, significa que podemos linearizar os vértices respeitando todas as dependencies — logo não há ciclo possível (pois um ciclo violaria essa ordem).
- Se há ciclo $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$, então qual é a ordem? v_1 deve vir antes de v_2 (por $v_1 \rightarrow v_2$), v_2 antes de v_3 , ..., mas v_k deve vir antes de



v_1 (por $v_k \rightarrow v_1$). Isso cria uma contradição: v_1 não pode estar antes e depois de si mesmo. Logo, nenhuma ordem linear é válida.

1.3. **Aplicações Práticas de DAGs e Ordenação Topológica:** Sistemas reais frequentemente usam DAGs em suas operações.

- Descreva uma aplicação de ordenação topológica em **gerenciamento de projetos** (ex: construção de estante ou software).
- Qual seria o resultado (impacto) se não houvesse a garantia de que o grafo de dependências é um DAG?

- Em gerenciamento de projetos (Project Scheduling), cada tarefa é um vértice e uma aresta $u \rightarrow v$ significa que a tarefa u deve ser concluída antes de v começar. Aplicar ordenação topológica fornece uma sequência válida de execução de tarefas. Exemplo: Construir estante: comprar tábuas (A) $\rightarrow$ cortar tábuas (B) $\rightarrow$ pintar (C) $\rightarrow$ montar (D) $\rightarrow$ transportar (E). Uma ordenação válida: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$.
- Se não fosse garantido DAG (ou seja, se houvesse ciclos), a ordenação topológica falharia e o sistema não conseguiria definir uma sequência viável. Em software, isso poderia resultar em deadlock (esp. em gerenciadores de recursos). Em construção, indicaria requerimentos contraditórios/impossíveis, sinalizando erro no planejamento.

1.4. **Deteção de Ciclos em Grafos Direcionados:** Uma forma de verificar se um grafo é DAG é tentar detectar ciclos.

- Descreva como usar **DFS** para detectar ciclos em um grafo direcionado. Qual é a condição que sinaliza a presença de um ciclo?
- Por que a detecção de aresta de “back edge” (aresta para um vértice já visitado no caminho atual) indica ciclo?

- Na DFS, classifica-se os vértices como “branco” (não visitado), “cinza” (visitado mas DFS ainda em execução) ou “preto” (DFS completada). Se durante a DFS encontrarmos uma aresta para um vértice **cinza**, há ciclo (retorno para um vértice no caminho atual). Usar timestamp: se para toda aresta $u \rightarrow v$, $\text{finish}(u) > \text{finish}(v)$, é DAG. Se existe $u \rightarrow v$ com $\text{finish}(u) < \text{finish}(v)$, há ciclo (back edge).
- Uma back edge $u \rightarrow v$ (aresta para um ancestral em DFS) cria um caminho de v até u (via predecessores na árvore DFS) e depois volta de u para v via back edge. Isso forma um ciclo: $v \rightarrow \dots \rightarrow u \rightarrow v$.



2 Ordenação Topológica: Algoritmo de Kahn

2.1. **Algoritmo de Kahn: Intuição Baseada em Grau de Entrada:** O algoritmo de Kahn é uma alternativa a DFS para encontrar ordenação topológica.

- Qual é a ideia central de Kahn? Em que conceito ela se baseia?
- Por que sempre há pelo menos um vértice com grau de entrada zero em um DAG?

- A ideia é: **sempre escolha quem não depende de ninguém.** Um vértice com grau de entrada zero (in-degree = 0) não tem predecessores e portanto pode ser processado imediatamente. Após processar esse vértice, remove-se suas arestas de saída, reduzindo o grau de entrada dos vizinhos. Novos vértices com grau zero ficam prontos.
- (Prova informal) Se não houvesse nenhum vértice com grau zero, todo vértice teria pelo menos um predecessor. Seguindo predecessores indefinidamente, eventualmente revisitaríamos um vértice (pigeonhole principle), formando um ciclo. Mas o grafo é DAG, o que é uma contradição. Logo, deve haver pelo menos um vértice com grau zero.

2.2. **Exemplo Passo a Passo: Kahn em um DAG:** Aplicar Kahn ao grafo de construção de estante.

- Considere o DAG com vértices $\{A, B, C, D, E, F\}$ e arestas: $A \rightarrow C$, $C \rightarrow D$, $B \rightarrow E$, $D \rightarrow E$, $E \rightarrow F$. Monte a tabela inicial de graus de entrada.
- Execute Kahn passo a passo via tabela, mostrando o nó processado, os graus atualizados, a fila e a lista topológica resultante.

- Graus Iniciais:** $A:0$, $B:0$, $C:1$, $D:1$, $E:2$, $F:1$. Como A e B possuem grau 0, iniciam na fila.

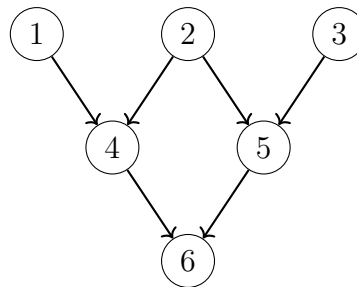
- Tabela de Rastreio (Kahn):**

Fase	Nó Retirado	Graus Reduzidos	Nova Fila Q	Ordem L
0	-	-	$[A, B]$	$[]$
1	A	C cai para 0 (entra na fila)	$[B, C]$	$[A]$
2	B	E cai para 1	$[C]$	$[A, B]$
3	C	D cai para 0 (entra na fila)	$[D]$	$[A, B, C]$
4	D	E cai para 0 (entra na fila)	$[E]$	$[A, B, C, D]$
5	E	F cai para 0 (entra na fila)	$[F]$	$[A, B, C, D, E]$
6	F	-	$[]$	$[A, B, C, D, E, F]$

A ordenação produzida é $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$. (B poderia ser processado antes de A, o que geraria outra ordem topológica válida).



2.3. **Prática de Fixação: Kahn (Resolvido à mão).** Considere o seguinte diagrama de pré-requisitos:

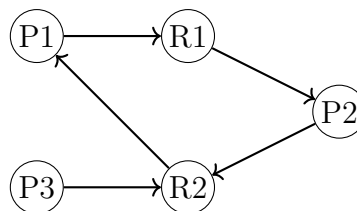


- a. Execute o algoritmo de Kahn iterativamente. Use ordem crescente numérica em caso de empate na Fila (ex: entre 1, 2 e 3, escolha o menor). Exiba a tabela com a Ordem Topológica gerada.

a. Graus iniciais: 1:0, 2:0, 3:0, 4:2, 5:2, 6:2. Fila Inicial: [1, 2, 3].

Fase	Nó	Graus Reduzidos	Fila Q	Ordem L
0	-	-	[1, 2, 3]	[]
1	1	Grau do 4 cai para 1.	[2, 3]	[1]
2	2	Grau do 4 cai para 0. Grau do 5 cai para 1.	[3, 4]	[1, 2]
3	3	Grau do 5 cai para 0.	[4, 5]	[1, 2, 3]
4	4	Grau do 6 cai para 1.	[5]	[1, 2, 3, 4]
5	5	Grau do 6 cai para 0.	[6]	[1, 2, 3, 4, 5]
6	6	-	[]	[1, 2, 3, 4, 5, 6]

2.4. **Prática Kahn: Falha por Deadlock (Detecção de Ciclo).** Considere o grafo direcionado abaixo modelando a alocação de recursos em um Sistema Operacional:



- a. Calcule os Graus de Entrada Iniciais. Qual nó inicia na Fila do algoritmo de Kahn?
- b. Faça o rastreo de mesa e mostre como e onde o algoritmo de Kahn “trava”, evidenciando a presença de um Deadlock. Indique quais vértices fazem parte do ciclo.

a. **Graus Iniciais:** P1:1, R1:1, P2:1, R2:2, P3:0. O único nó com grau 0 é o **P3**, logo a Fila inicia com [P3].



b. Tabela de Rastreio (Kahn):

Fase	Nó Retirado	Graus Reduzidos	Nova Fila Q	Ordem L
0	-	-	[P3]	[]
1	P3	R2 cai para 1. Nenhum nó novo atingiu grau 0.	[]	[P3]

Na Fase 1, a fila Q ficou vazia, mas a lista topológica só tem 1 elemento (falta P1, R1, P2, R2). O algoritmo **trava** (aborta), sinalizando **Ciclo (Deadlock)**.

Os nós restantes {P1, R1, P2, R2} estão em um ciclo fechado onde cada um depende do outro.

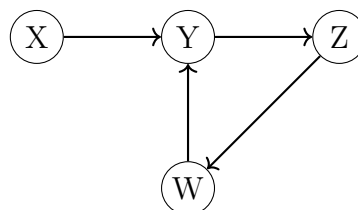
3 Ordenação Topológica: Abordagem via DFS

3.1. **Ordenação Topológica via DFS:** Uma alternativa ao algoritmo de Kahn é usar DFS.

- Como a DFS produz uma ordenação topológica? Qual é o critério?
- Qual é a relação entre “finishing time” (tempo de término) e a Ordenação Topológica?

- Executa-se DFS em todos os vértices (em qualquer ordem inicial). Para cada vértice, registra-se o “finishing time” (tempo em que a DFS recursiva termina). Ordena-se os vértices em ordem **decrecente** de finishing time. Essa ordem é uma ordenação topológica válida.
- Para toda aresta $u \rightarrow v$, a DFS garante que $\text{finish}(u) > \text{finish}(v)$. Isso porque quando DFS é chamado em u , ela explora v (se ainda não visitado), terminando a recursão em v antes de terminar em u . Logo, $\text{finish}(v) < \text{finish}(u)$. Ordenar por finishing time decrescente respeita todas as arestas.

3.2. **Prática DFS: Back-Edge em Grafo com Ciclo.** Considere o grafo direcionado a seguir:



- Rastreie uma DFS começando do vértice **X**. Use os estados “Não visitado”, “Em exploração (Cinza)” e “Finalizado (Preto)”.



Pontifícia Universidade Católica de Minas Gerais

Inst. de Ciências Exatas e Informática, Campus Lourdes

Curso de Engenharia de Computação

- b. Em que ponto exato a DFS detecta o ciclo? Como essa aresta é chamada?

a. **Rastreio de Estados:**

Tempo	Ação (Pilha de Recursão)	Nó Atual	Estado do Nó
1	Descoberta a partir do main	X	X muda para Cinza
2	X visita Y	Y	Y muda para Cinza
3	Y visita Z	Z	Z muda para Cinza
4	Z visita W	W	W muda para Cinza
5	W tenta visitar Y.	Y	Conflito! Y já é Cinza.

- b. No tempo 5, partindo de W, a recursão tenta visitar o vizinho Y. Como Y já está no estado “Em exploração” (Cinza), ou seja, ele está ativo na pilha de chamadas atual, o algoritmo percebe que retornou para um ancestral. A aresta ($W \rightarrow Y$) é uma **Back-Edge (Aresta de Retorno)**. Isso comprova a existência de um ciclo e aborta a Ordenação Topológica.

3.3. **Exemplo: DFS para Ordenação Topológica:** Considere o mesmo DAG $\{A, B, C, D, E, F\}$ com arestas $A \rightarrow C, C \rightarrow D, B \rightarrow E, D \rightarrow E, E \rightarrow F$.

- a. Execute a DFS (iniciando pelos vértices em ordem alfabética). Se o vértice já foi visitado, pule. Registre os tempos de descoberta (D) e finalização (F).
b. Monte a ordenação topológica classificando em ordem decrescente de F .

- a. O relógio (tempo) começa em 1. A DFS explora todo ramo até encontrar vértices sem saída (sumidouros) ou já visitados.

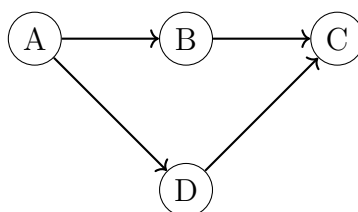
Tempo	Ação (Pilha de Recursão)	Nó	D/F
1	Descoberta a partir de main	A	D(A)=1
2	A visita C	C	D(C)=2
3	C visita D	D	D(D)=3
4	D visita E	E	D(E)=4
5	E visita F	F	D(F)=5
6	F não tem vizinhos. Finaliza.	F	F(F)=6
7	Volta para E. Sem mais vizinhos.	E	F(E)=7
8	Volta para D. Finaliza.	D	F(D)=8
9	Volta para C. Finaliza.	C	F(C)=9
10	Volta para A. Finaliza.	A	F(A)=10
11	main chama prox. não visitado: B.	B	D(B)=11
12	B aponta p/ E (já visitado). Finaliza.	B	F(B)=12

- b. Os tempos de finalização são: B(12), A(10), C(9), D(8), E(7), F(6).

Ordem decrescente: $B \rightarrow A \rightarrow C \rightarrow D \rightarrow E \rightarrow F$.

Esta ordem é válida e diferente do Kahn clássico (A, B, C, D, E, F), mas ambas respeitam perfeitamente as dependências.

3.4. **Prática de Fixação: DFS (Resolvido à mão).** Considere o seguinte DAG de pacotes de software e suas dependências (seta indica “depende de”):



- a. Execute a DFS começando pelo nó **A**. Em caso de empate, visite os vizinhos em ordem alfabética. Registre o relógio (D/F) e extraia a ordenação topológica final (inversa do tempo de finalização).

a. Tabela de Rastreio da DFS:

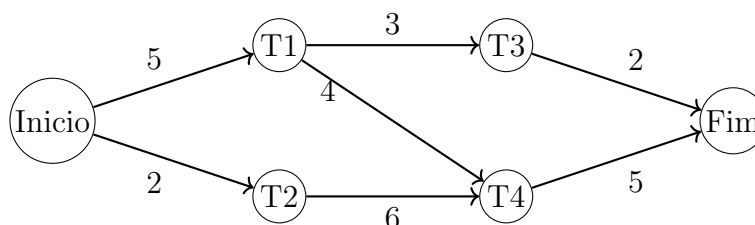
Tempo	Ação (Pilha de Recursão)	Nó	D/F
1	Descoberta a partir do main	A	D(A)=1
2	A visita B (alfabética)	B	D(B)=2
3	B visita C	C	D(C)=3
4	C não tem vizinhos. Finaliza.	C	F(C)=4
5	Volta para B. Sem vizinhos. Finaliza.	B	F(B)=5
6	Volta para A. A visita D.	D	D(D)=6
7	D aponta para C (já visitado). Finaliza.	D	F(D)=7
8	Volta para A. Finaliza.	A	F(A)=8

Tempos de finalização (F): A(8), D(7), B(5), C(4).

Ordenação Topológica (Decrescente): **A** → **D** → **B** → **C**. (Note que A → B → D → C também seria válida se D fosse visitado antes de B).

4 Aplicações Avançadas e Análise

- 4.1. **Caminho Crítico (Critical Path):** Uma aplicação essencial de DAGs ponderados é determinar o Caminho Crítico em cronogramas de projetos. Considere o DAG abaixo, onde os vértices representam tarefas e os pesos nas arestas representam o custo (duração em dias) para transitar de uma tarefa à outra:



- a. Calcule e informe qual é o tamanho (duração total) do Caminho Crítico (o caminho **mais longo** do Inicio ao Fim).
- b. Descreva a sequência de vértices que compõem este caminho crítico.



a. Listando todos os caminhos possíveis do **Início** ao **Fim**:

- Início $\rightarrow$ T1 $\rightarrow$ T3 $\rightarrow$ Fim: $5 + 3 + 2 = 10$ dias.
- Início $\rightarrow$ T1 $\rightarrow$ T4 $\rightarrow$ Fim: $5 + 4 + 5 = 14$ dias.
- Início $\rightarrow$ T2 $\rightarrow$ T4 $\rightarrow$ Fim: $2 + 6 + 5 = 13$ dias.

A duração total do Caminho Crítico é **14 dias**.

b. A sequência de tarefas é: **Início $\rightarrow$ T1 $\rightarrow$ T4 $\rightarrow$ Fim**.

4.2. **Reachability e Closure Transitivo em DAGs:** Outro problema clássico em DAGs.

- Defina o **fecho transitivo** (transitive closure) de um grafo.
- Qual é a importância do fecho transitivo em análise de dependências?

a. O **fecho transitivo** de um grafo $G = (V, E)$ é o grafo $G^* = (V, E^*)$ onde $(u, v) \in E^*$ se e somente se existe um caminho de u para v em G (direto ou indireto). Para DAGs, G^* é importante porque identifica **todas** as dependências transitivas.

b. Exemplo: Se Disciplina $A \rightarrow B \rightarrow C$ (B depende de A, C depende de B), o fecho transitivo também adiciona $A \rightarrow C$ (C indiretamente depende de A). Em planejamento, isso ajuda a identificar todas as restrições, mesmo as indiretas. Pode ser computado via algoritmo de Floyd-Warshall (adaptado para DAGs) em $O(V^3)$, ou $O(V(V + E))$ com DFS para cada vértice em um DAG.

4.3. **Contagem de Ordenações Topológicas Válidas:** Nem todos os DAGs têm uma única ordenação topológica.

- Quando há múltiplas ordenações topológicas válidas para um DAG?
- Como calcular ou estimar o número de ordenações topológicas válidas?

a. Múltiplas ordenações existem quando há vértices que não estão em uma relação de ordem direta. Exemplo: se $A \rightarrow C$ e $B \rightarrow C$ (A e B não estão conectados), então $A \rightarrow B \rightarrow C$ e $B \rightarrow A \rightarrow C$ são ambas válidas.

b. Calcular o número exato de ordenações é #P-hard em geral. Para grafos simples, pode-se usar programação dinâmica ou enumeração recursiva. Um limite máximo é $|V|!$ (todas as permutações), e um limite mínimo é 1 (se há um caminho hamiltoniano no DAG).



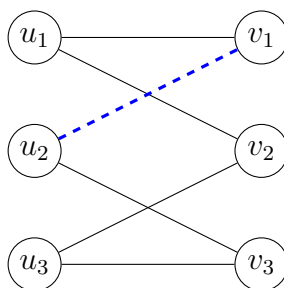
5 Emparelhamento (Matching) e Grafos Bipartidos

5.1. **Emparelhamento Maximal, Máximo e Casamento Perfeito:** Compreender a diferença entre essas classificações é essencial.

- Defina formalmente o que é um **Emparelhamento** (Matching) em um grafo $G = (V, E)$.
- Diferencie um Emparelhamento **Maximal** de um Emparelhamento **Máximo**. Dê um exemplo (ou explique) de um grafo onde um emparelhamento maximal **NÃO** é máximo.
- O que é um **Casamento Perfeito** (1-fator)? Todo casamento perfeito é um emparelhamento máximo?

- Um **emparelhamento** (ou matching) é um subconjunto de arestas $M \subseteq E$ tal que não existem duas arestas em M que compartilhem um mesmo vértice. (Ou seja, cada vértice de V incide em no máximo uma aresta de M).
- Um emparelhamento **maximal** é aquele ao qual não se pode adicionar nenhuma aresta sem violar a restrição de emparelhamento (não é subconjunto próprio de outro emparelhamento). Um emparelhamento **máximo** é aquele que contém o maior número possível de arestas para aquele grafo. Exemplo: Um caminho $A - B - C - D$. O conjunto $M_1 = \{(B, C)\}$ é maximal (não posso adicionar (A, B) nem (C, D) pois incidem em B e C), mas não é máximo. O conjunto $M_2 = \{(A, B), (C, D)\}$ é máximo (tamanho 2).
- Um **casamento perfeito** é um emparelhamento que cobre todos os vértices do grafo (tamanho $|V|/2$). Sim, todo casamento perfeito é automaticamente um emparelhamento máximo, pois é impossível ter mais arestas do que a metade do número de vértices. Mas nem todo emparelhamento máximo é perfeito.

5.2. **Prática: Caminhos Aumentantes em Grafo Bipartido.** Considere o grafo bipartido $G = (U \cup V, E)$ abaixo, onde a linha tracejada e espessa representa o emparelhamento inicial M :



- O que é um **caminho alternante** e um **caminho aumentante**?



- O emparelhamento inicial $M = \{(u_2, v_1)\}$ é maximal ou máximo? Identifique os vértices livres.
- Encontre um caminho aumentante partindo de u_1 e aplique a operação de diferença simétrica para aumentar o emparelhamento. Qual o novo M ?
- O novo emparelhamento é um casamento perfeito?

- Um **caminho alternante** é um caminho cujas arestas alternam entre pertencer a M e não pertencer a M . Um **caminho aumentante** é um caminho alternante que começa e termina em vértices **livres** (não cobertos por M).
- $M = \{(u_2, v_1)\}$ não é maximal, pois podemos adicionar (u_3, v_3) sem conflito. Os vértices livres em U são $\{u_1, u_3\}$ e em V são $\{v_2, v_3\}$.
- Iniciando no vértice livre u_1 , buscamos um caminho alternante.

Sequência do Caminho	Status da Aresta
$u_1 \rightarrow v_1$	Fora de M (Nova)
$v_1 \rightarrow u_2$	Dentro de M (Existente)
$u_2 \rightarrow v_3$	Fora de M (Nova)

Como v_3 também é livre, temos um **caminho aumentante**.

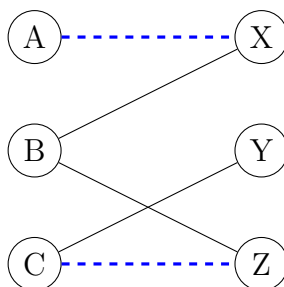
Aplicamos a **Diferença Simétrica** ($M \oplus P$):

- Removemos: (u_2, v_1) .
- Adicionamos: (u_1, v_1) e (u_2, v_3) .

Novo emparelhamento: $M' = \{(u_1, v_1), (u_2, v_3)\}$.

- Ainda não. Os vértices u_3 e v_2 continuam livres e existe uma aresta os ligando. O caminho aumentante trivial $u_3 \rightarrow v_2$ aumenta o emparelhamento em mais 1. Resultado final: $M'' = \{(u_1, v_1), (u_2, v_3), (u_3, v_2)\}$. Como o tamanho é 3 (cobrindo todos os $|V| = 6$ vértices), atingimos um **casamento perfeito**.

5.3. **Prática Hopcroft-Karp 1 (Aumento Simples).** Considere o grafo bipartido abaixo. As linhas espessas em azul representam o emparelhamento inicial $M = \{(A, X), (C, Z)\}$.

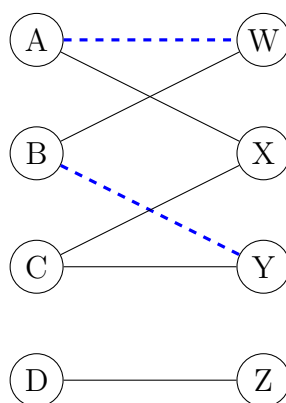




- Quais são os vértices livres?
- Encontre um caminho alternante que ligue dois vértices livres (caminho aumentante). Descreva a sequência.
- Aplice a operação de Diferença Simétrica e liste o novo emparelhamento máximo.

- Vértices livres: **B** e **Y**.
- O caminho deve começar no livre B e terminar no livre Y, alternando arestas fora de M e dentro de M. O único caminho válido é: $B \rightarrow Z$ (Fora), $Z \rightarrow C$ (Dentro), $C \rightarrow Y$ (Fora).
- Arestas Removidas: (C, Z) . Arestas Adicionadas: (B, Z) e (C, Y) . O novo emparelhamento será $M' = \{(A, X), (B, Z), (C, Y)\}$.

5.4. **Prática Hopcroft-Karp 2 (Múltiplos Caminhos).** Neste cenário, temos um emparelhamento inicial $M = \{(A, W), (B, Y)\}$ representado pela linha espessa azul.



- Quais são os vértices livres? Este emparelhamento inicial é maximal?
- Encontre um caminho aumentante partindo de **C**. Liste as arestas, faça a diferença simétrica e descreva o novo conjunto de emparelhamento M_1 .
- Ainda restam vértices livres. Encontre mais um caminho aumentante e calcule o emparelhamento máximo final M_{final} .

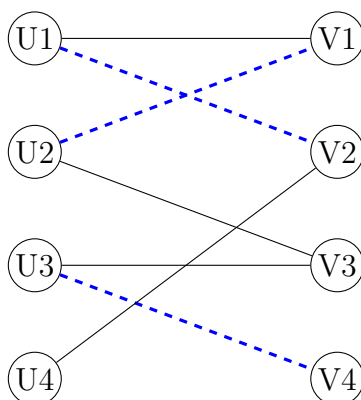
- Vértices livres: $\{C, D\}$ e $\{X, Z\}$. O emparelhamento não é maximal, pois D e Z possuem aresta direta sem usar vértices emparelhados.
- Partindo de C, o caminho até um nó livre em V é: $C \rightarrow Y$ (Fora), $Y \rightarrow B$ (Dentro), $B \rightarrow W$ (Fora), $W \rightarrow A$ (Dentro), $A \rightarrow X$ (Fora). Caminho aumentante válido. Diferença Simétrica: removemos (B, Y) e (A, W) . Inserimos (C, Y) , (B, W) e (A, X) . Novo emparelhamento: $M_1 = \{(C, Y), (B, W), (A, X)\}$.



- c. Os livres restantes são D e Z. O caminho aumentante é a própria aresta direta: $D \rightarrow Z$. Adicionando (D, Z) a M_1 , temos o casamento perfeito final: $M_{final} = \{(C, Y), (B, W), (A, X), (D, Z)\}$.

5.5. **Prática Hopcroft-Karp 3 (Grafo Denso).** A rede bipartida abaixo contém múltiplas opções de conexão. O emparelhamento atual é:

$$M = \{(U_1, V_2), (U_2, V_1), (U_3, V_4)\}.$$



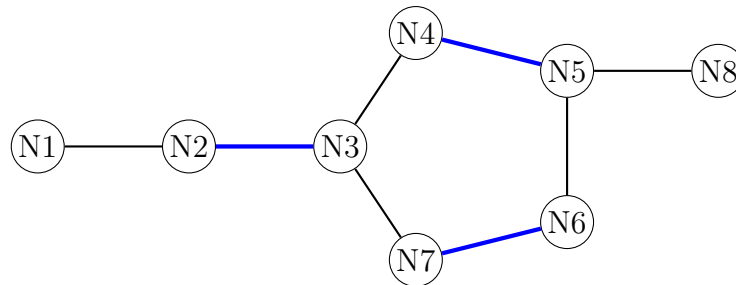
- a. Liste os vértices livres. Tentar conectar diretamente $U_4 \rightarrow V_3$ formaria um caminho aumentante válido? Explique usando a definição de caminho alternante.
- b. Desenvolva o único caminho aumentante válido que começa em U_4 . Liste a sequência de nós.
- c. Após aplicar o caminho aumentante encontrado na letra b, liste as tuplas do emparelhamento M' .

- a. Vértices livres: U_4 e V_3 . A ligação direta $U_4 \rightarrow V_3$ **não é válida** porque simplesmente não existe aresta entre U_4 e V_3 no grafo original (veja o diagrama). O caminho precisa trafegar por arestas existentes.
- b. Partindo do nó livre U_4 , a única aresta disponível é para V_2 (Fora de M). De V_2 , seguimos pela aresta espessa (Dentro de M) para U_1 . De U_1 , seguimos para V_1 (Fora de M). De V_1 , seguimos para U_2 (Dentro de M). De U_2 , finalmente alcançamos o nó livre V_3 (Fora de M). O caminho aumentante é: $U_4 \rightarrow V_2 \rightarrow U_1 \rightarrow V_1 \rightarrow U_2 \rightarrow V_3$.
- c. Diferença Simétrica: Removemos as espessas contidas no caminho: (U_1, V_2) e (U_2, V_1) . Inserimos as outras do caminho (U_4, V_2) , (U_1, V_1) , (U_2, V_3) . A aresta de (U_3, V_4) fica intacta. O novo emparelhamento máximo é: $M' = \{(U_4, V_2), (U_1, V_1), (U_2, V_3), (U_3, V_4)\}$.

5.6. **Matching em Grafos Gerais: Algoritmo Blossom (Edmonds).** Em grafos não-bipartidos, a presença de ciclos de tamanho ímpar (Blossoms) faz com que a



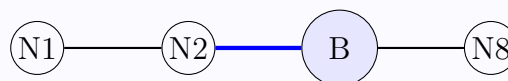
busca simples por caminhos alternantes falhe. O algoritmo de Edmonds resolve isso contraindo esses ciclos em um "supervértice".



Estado atual: As arestas grossas/azuis pertencem ao emparelhamento M . Os nós N1 e N8 estão **livres**.

- Ao iniciar a busca a partir de N1, uma "flor" (Blossom) é detectada devido à presença do ciclo ímpar. Quais nós compõem a flor e qual é o nó "Base" (raiz da flor)? Por que ele é a base?
- Contraindo a flor inteira em um único supervértice B , qual é o caminho aumentante encontrado no grafo contraído indo de N1 até N8?
- Ao expandir o supervértice B de volta para os nós originais, detalhe o caminho aumentante final completo de N1 a N8, garantindo que ele alterne estritamente entre arestas fora e dentro de M .

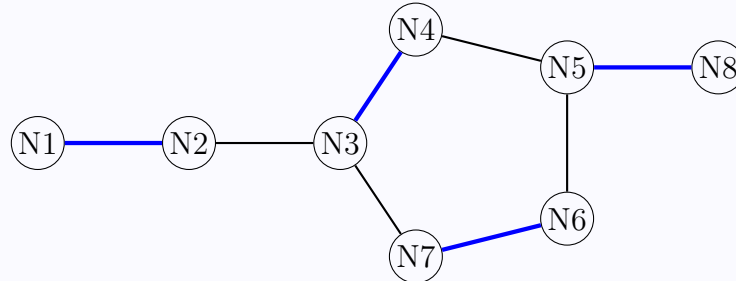
- A flor (ciclo ímpar) é formada pelos nós $\{N3, N4, N5, N6, N7\}$. O nó Base é $N3$, pois é o único nó do ciclo que está conectado ao resto do caminho (o "Stem") através de uma aresta que pertence ao emparelhamento M (a aresta grossa N2-N3).
- Contraindo a flor em um supervértice B , a aresta original N1-N2 continua intacta. A aresta N2-N3 vira N2- B (pertencente a M). A aresta N5-N8 vira B -N8 (fora de M). O caminho aumentante simples no grafo contraído é: $N1 \rightarrow N2 \rightarrow B \rightarrow N8$.



- O caminho **obrigatoriamente tem que percorrer a parte de dentro do blossom** para conectar a entrada (N3) até a saída (N5). A regra de ouro da expansão é: o ciclo ímpar oferece duas rotas de N3 até N5, e nós **devemos sempre escolher o caminho de tamanho par**. Isso garante que a alternância se mantenha intacta (começando e terminando com uma aresta fora de M). O caminho par tem 2 arestas: $N3$ (**fora**) $\rightarrow N4$ (**dentro**) $\rightarrow N5$. Juntando tudo, o caminho aumentante final expandido é: $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N8$. (Ele alterna perfeitamente: fora, dentro, fora, dentro, fora).



Aplicando a diferença simétrica (invertendo o status das arestas desse caminho), o novo emparelhamento máximo fica assim:



6 Casamento Estável: Algoritmo de Gale-Shapley

6.1. Gale-Shapley — Conceito de Casamento Estável:

- Defina o que caracteriza um emparelhamento como **estável**. O que é um “par bloqueador”?
- No algoritmo de Gale-Shapley (Proponentes e Revisores), qual dos dois grupos leva vantagem na alocação final? Por quê?

- Um emparelhamento é **estável** se não existe nenhum par (A, B) não casado entre si, mas onde **ambos** prefeririam estar um com o outro em vez de com seus parceiros atuais. Um par nessas condições é chamado de **par bloqueador**, pois eles teriam incentivo para abandonar seus pares e ficarem juntos, rompendo a estabilidade.
- O grupo dos **Proponentes** (quem faz as propostas) leva a vantagem sistêmica. O algoritmo garante a eles o melhor casamento estável possível (ótimo para proponentes), enquanto os **Revisores** acabam com o pior casamento estável possível para eles. Isso ocorre porque o proponente tenta de cima para baixo em sua lista de preferência (ordem decrescente), enquanto o revisor só pode aceitar passivamente o “menos pior” que lhe foi proposto.

- 6.2. Gale-Shapley — Simulação Passo a Passo (Cascata): Considere 3 Hospitais (Proponentes) e 3 Médicos (Revisores) com as matrizes de preferência (do mais para o menos preferido):

Hospitais	1ª	2ª	3ª
H_1	M_1	M_2	M_3
H_2	M_2	M_1	M_3
H_3	M_1	M_2	M_3

Médicos	1ª	2ª	3ª
M_1	H_2	H_3	H_1
M_2	H_1	H_2	H_3
M_3	H_1	H_2	H_3

Execute o algoritmo iterativamente. Mostre as propostas feitas e a alocação final.



Pontifícia Universidade Católica de Minas Gerais

Inst. de Ciências Exatas e Informática, Campus Lourdes

Curso de Engenharia de Computação

Rodada	Propostas Feitas	Decisões dos Revisores (Médicos)
1	H_1 propõe a M_1 . H_2 propõe a M_2 . H_3 propõe a M_1 .	M_1 aceita H_1 (provisório). M_2 aceita H_2 (provisório). M_1 recebe H_3 . Prefere H_3 a H_1 . Rejeita H_1 , aceita H_3 .
2	H_1 (livre) propõe a M_2 .	M_2 recebe H_1 . Prefere H_1 a H_2 . Rejeita H_2 , aceita H_1 .
3	H_2 (livre) propõe a M_1 .	M_1 recebe H_2 . Prefere H_2 a H_3 . Rejeita H_3 , aceita H_2 .
4	H_3 (livre) propõe a M_2 . H_3 (livre) propõe a M_3 .	M_2 prefere H_1 a H_3 . Rejeita H_3 . M_3 aceita H_3 (provisório).

Não há mais proponentes livres. O casamento estável final é:

$\{(H_1, M_2), (H_2, M_1), (H_3, M_3)\}$.

(Cascata: $H_3 \rightarrow M_1$, $H_1 \rightarrow M_2$, $H_2 \rightarrow M_1$, $H_3 \rightarrow M_3$).

6.3. **Gale-Shapley — Simulação Estendida (4x4):** Considere 4 Empresas (E_1 a E_4 , proponentes) e 4 Candidatos (C_1 a C_4 , revisores). Matrizes de preferência:

Empresa	1ª	2ª	3ª	4ª
E1	C1	C2	C3	C4
E2	C2	C1	C4	C3
E3	C1	C3	C2	C4
E4	C1	C4	C2	C3

Candidato	1ª	2ª	3ª	4ª
C1	E2	E3	E4	E1
C2	E3	E1	E2	E4
C3	E1	E4	E2	E3
C4	E1	E2	E3	E4

Preencha a tabela de rastreamento de propostas e encontre o casamento final.

Rodada	Propostas das Empresas (Livre tenta o próximo)	Decisão dos Candidatos (Aceita se melhor)
1	$E_1 \rightarrow C_1$ $E_2 \rightarrow C_2$ $E_3 \rightarrow C_1$ $E_4 \rightarrow C_1$	C_2 aceita E_2 . C_1 recebe E_1, E_3, E_4 . A preferência de C_1 é $E_3 \succ E_4 \succ E_1$. Então C_1 aceita E_3 e rejeita E_1 e E_4 .
2	E_1 (livre) $\rightarrow C_2$ E_4 (livre) $\rightarrow C_4$	C_4 aceita E_4 . C_2 recebe E_1 . C_2 prefere E_1 ao atual E_2 . C_2 aceita E_1 e rejeita E_2 .
3	E_2 (livre) $\rightarrow C_1$	C_1 recebe E_2 . C_1 prefere E_2 ao atual E_3 . C_1 aceita E_2 e rejeita E_3 .
4	E_3 (livre) $\rightarrow C_3$	C_3 aceita E_3 .

Não há mais empresas livres. Casamento Estável: $\{(E_1, C_2), (E_2, C_1), (E_3, C_3), (E_4, C_4)\}$.

7 Emparelhamento Ponderado: Método Húngaro

7.1. **Método Húngaro — Fundamentos:** O Método Húngaro resolve o problema de alocação ótima em grafos bipartidos ponderados.



- Qual a diferença entre o matching clássico e o ponderado? O que o Húngaro minimiza?
- Como aplicar o Método Húngaro se o problema for de **maximização**?

- No matching clássico busca-se maximizar a quantidade de pares. No **matching ponderado**, cada aresta tem um peso/custo. O objetivo é achar um casamento perfeito onde a **soma dos custos seja a menor possível**.
- Para maximização, subtraia os valores da matriz do valor máximo presente. Depois, aplique o Húngaro na matriz resultante.

7.2. **Método Húngaro — Passo a Passo:** 4 Tarefas (T1 a T4) para 4 Funcionários (F1 a F4). Matriz de custos:

	T1	T2	T3	T4
F1	10	19	8	15
F2	10	18	7	17
F3	13	16	9	14
F4	12	19	8	18

Aplique os passos do Método Húngaro para encontrar a alocação de menor custo.

Passo 1: Redução de Linha (Subtrair o menor valor de cada linha). Mínimos: L1(8), L2(7), L3(9), L4(8). Resulta em:

	T1	T2	T3	T4
F1	2	11	0	7
F2	3	11	0	10
F3	4	7	0	5
F4	4	11	0	10

Passo 2: Redução de Coluna (Subtrair menor valor de cada coluna). Mínimos: C1(2), C2(7), C3(0), C4(5). Resulta em:

	T1	T2	T3	T4
F1	0	4	0	2
F2	1	4	0	5
F3	2	0	0	0
F4	2	4	0	5

Passo 3: 1ª Cobertura. Zeros em (F1,T1), (F1,T3), (F2,T3), (F3,T2), (F3,T3), (F3,T4), (F4,T3). Podemos cobrir todos os zeros com 3 retas: Linha F1, Linha F3, e Coluna T3. Como $3 < 4$, ajustamos. (Menor valor não coberto é 1 em F2,T1). Subtraímos 1 dos livres, somamos 1 nos cruzamentos (F1,T3 e F3,T3).



	T1	T2	T3	T4
F1	0	4	1	2
F2	0	3	0	4
F3	2	0	1	0
F4	1	3	0	4

Passo 4: 2ª Cobertura. Zeros em (F1,T1), (F2,T1), (F2,T3), (F3,T2), (F3,T4), (F4,T3). Ainda podemos cobrir com 3 retas! Coluna T1, Coluna T3, e Linha F3. Como $3 < 4$, ajustamos novamente. (Menor valor livre é 2 em F1,T4). Subtraímos 2 dos livres, somamos 2 nos cruzamentos (F3,T1 e F3,T3).

	T1	T2	T3	T4
F1	0	2	1	0
F2	0	1	0	2
F3	4	0	3	0
F4	1	1	0	2

Passo 5: Alocação. F4 só tem T3 → F4 alocado a T3. Sobra T1 para F2 → F2 alocado a T1. Sobra T4 para F1 → F1 alocado a T4. Sobra T2 para F3 → F3 alocado a T2.

Custo Mínimo: F1-T4 (15) + F2-T1 (10) + F3-T2 (16) + F4-T3 (8) = **49**.

7.3. **Teoria: Algoritmo do Método Húngaro.** O Método Húngaro resolve em tempo polinomial o problema clássico de assinalamento. Escreva em formato de lista (pseudocódigo em alto nível) os 4 passos principais do algoritmo, considerando a minimização.

Pseudocódigo (Alto nível):

1. **Redução de Linha:** Encontre o menor elemento de cada linha. Subtraia esse mínimo de todos os elementos da respectiva linha.
2. **Redução de Coluna:** Na matriz resultante, encontre o menor elemento de cada coluna. Subtraia esse mínimo de todos os elementos da respectiva coluna.
3. **Cobertura de Zeros:** Trace o número **mínimo** possível de linhas horizontais e verticais de modo a cobrir todos os zeros da matriz.
 - Se o número de retas traçadas for igual a N (dimensão da matriz), vá para o passo de **Alocação** (passo 4).
 - Se for menor que N , proceda com o **Ajuste de Custo**. Encontre o menor elemento não coberto por nenhuma reta. Subtraia este elemento de todos os elementos não cobertos. Adicione este elemento aos valores posicionados no **cruzamento** de duas retas. Volte ao passo 3.



Pontifícia Universidade Católica de Minas Gerais

Inst. de Ciências Exatas e Informática, Campus Lourdes

Curso de Engenharia de Computação

4. **Alocação:** Assinale os zeros, priorizando linhas/colunas que possuam apenas 1 zero. Remova a linha e coluna do zero selecionado e repita até alocar N pares.

- 7.4. **Prática Húngaro 1 (Minimização Simples 3x3).** Considere a matriz de custos de 3 trabalhadores para 3 máquinas:

	M1	M2	M3
T1	250	400	350
T2	400	600	350
T3	200	400	250

- a. Execute o Método Húngaro. Mostre a matriz após redução de linha e coluna. É necessário fazer ajuste de cobertura ou a primeira tentativa já alcança $N = 3$ retas?
- b. Liste a alocação ótima e a soma do custo mínimo.

- a. **Redução Linhas:** Mínimos são $L1=250$, $L2=350$, $L3=200$.

	M1	M2	M3
T1	0	150	100
T2	50	250	0
T3	0	200	50

Redução Colunas: Mínimos são $C1=0$, $C2=150$, $C3=0$.

	M1	M2	M3
T1	0	0	100
T2	50	100	0
T3	0	50	50

Cobertura de zeros: Podemos cobrir todos os zeros com 3 retas, por exemplo: Linha 1, Linha 2 e Coluna 1. Como 3 retas igualam a dimensão $N = 3$, a matriz está pronta. **Não é necessário ajuste.**

- b. Alocação: T2 obrigatoriamente pega M3. Sobra M1 e M2 para T1 e T3. T3 só tem zero em M1, então pega M1. Sobra M2 para T1. Alocação Ótima: **T1-M2, T2-M3, T3-M1**. Custo Total = $400 + 350 + 200 = 950$.

- 7.5. **Prática Húngaro 2 (Ajuste de Zeros 4x4).** Considere a matriz de custos a seguir, já após ter sofrido a redução de linhas e colunas:

	Col1	Col2	Col3	Col4
L1	0	5	8	3
L2	0	4	7	2
L3	2	0	3	0
L4	5	3	0	1



Pontifícia Universidade Católica de Minas Gerais

Inst. de Ciências Exatas e Informática, Campus Lourdes

Curso de Engenharia de Computação

- Tente cobrir os zeros. Quantas retas são necessárias?
- Como o número de retas é menor que 4, realize a operação de ajuste usando o menor elemento não coberto. Exiba a nova matriz após o ajuste.

a. Zeros em (L1,C1), (L2,C1), (L3,C2), (L3,C4), (L4,C3). Podemos cobrir todos eles usando apenas **3 retas**: Coluna 1, Linha 3, e Linha 4 (ou Coluna 1, Coluna 2 e Coluna 3 - mas com cruzamentos diferentes. Usaremos a padrão horizontal/vertical mais abrangente: C1, L3, L4).

b. Os elementos livres não cobertos por essas 3 retas são:
(L1,C2)=5, (L1,C3)=8, (L1,C4)=3, (L2,C2)=4, (L2,C3)=7, (L2,C4)=2.
O menor elemento livre é **2**. Subtraímos 2 de todos os livres. Somamos 2 nos cruzamentos das retas (que são L3-C1 e L4-C1). O restante das células cobertas continua igual.

Nova matriz:

	Col1	Col2	Col3	Col4
L1	0	3	6	1
L2	0	2	5	0
L3	4 (cruz.)	0	3	0
L4	7 (cruz.)	3	0	1

(Agora, a matriz requer 4 retas para cobrir os zeros).

7.6. Prática Húngaro 3 (Maximização de Lucro 4x4). A tabela apresenta o lucro esperado (em milhares de reais) que 4 corretores geram em 4 diferentes regiões de vendas. Deseja-se maximizar o lucro total.

	R1	R2	R3	R4
C1	40	20	30	50
C2	20	60	40	30
C3	30	40	50	20
C4	50	30	20	40

- Converta o problema de maximização para minimização extraindo a matriz da diferença. (Dica: Ache o maior valor de todos e subtraia toda a matriz dele).
- Resolva o Húngaro passo a passo nesta nova matriz e determine a alocação que trará o lucro máximo total para a empresa. Indique qual é esse lucro.

a. O valor máximo da matriz é **60**. A nova matriz de "custo/perda" será calculada por $(60 - X)$:

	R1	R2	R3	R4
C1	20	40	30	10
C2	40	0	20	30
C3	30	20	10	40
C4	10	30	40	20



b. **Redução de Linha:** L1(-10), L2(-0), L3(-10), L4(-10).

	R1	R2	R3	R4
C1	10	30	20	0
C2	40	0	20	30
C3	20	10	0	30
C4	0	20	30	10

Redução de Coluna: Os mínimos das colunas são todos zero. A matriz já está pronta! **Cobertura:** Temos zeros isolados e distribuídos de forma que não é possível cobrir com menos de 4 retas. **Alocação Direta:** C4 pega R1 (único zero de C4 e R1). C2 pega R2. C3 pega R3. C1 pega R4. **Lucro Ótimo (Voltando aos valores originais):** C1(R4)=50, C2(R2)=60, C3(R3)=50, C4(R1)=50. Soma Total = 50 + 60 + 50 + 50 = **210** milhares de reais.

8 Planaridade de Grafos

8.1. Grafos Planares e a Fórmula de Euler:

- O que define um grafo como planar? O que são as “faces” de um grafo planar desenhado?
- Enuncie a Fórmula de Euler para grafos planares conexos.
- Dado um grafo planar conexo com $V = 6$ vértices e $E = 9$ arestas, quantas faces F esse grafo possuirá em qualquer representação planar?

- Um grafo é planar se puder ser desenhado no plano de forma que nenhuma de suas arestas se cruze (exceto nos vértices onde incidem). Uma “face” é uma região do plano delimitada por arestas do grafo. O desenho sempre inclui a face externa (região infinita ao redor do grafo).
- A Fórmula de Euler é: $V - E + F = 2$, onde V é o número de vértices, E é o número de arestas e F é o número de faces.
- Substituindo na fórmula: $6 - 9 + F = 2 \Rightarrow -3 + F = 2 \Rightarrow F = 5$. O grafo possuirá 5 faces (incluindo a face externa).

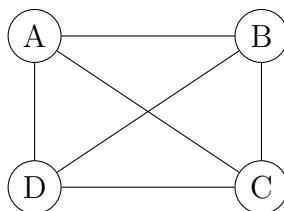
8.2. Teorema de Kuratowski:

- Qual é o enunciado do Teorema de Kuratowski?
- Descreva brevemente o que são os grafos K_5 e $K_{3,3}$. Por que eles são fundamentais no estudo de planaridade?



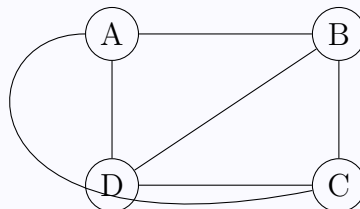
- O Teorema de Kuratowski afirma que um grafo finito é planar se, e somente se, ele **não contém** nenhum subgrafo que seja isomorfo a, ou uma **subdivisão** de, K_5 ou $K_{3,3}$.
- K_5 é o grafo completo de 5 vértices (todo vértice se conecta a todos os outros). $K_{3,3}$ é o grafo bipartido completo com 3 vértices em cada partição (o problema clássico das "3 casas e 3 utilidades"). Ambos são os grafos não planares com o menor número de arestas possível para sua categoria. Eles são as sementes da não-planaridade: qualquer grafo não planar esconde em sua estrutura um desses dois padrões.

8.3. **Prática: Comprovando a Planaridade.** Considere o grafo desenhado abaixo, com cruzamentos visíveis:



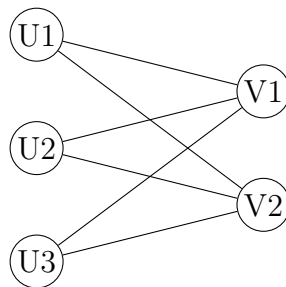
- Identifique o grafo acima. Ele possui representação planar?
- Em caso afirmativo, desenhe uma representação isomorfa sem cruzamentos.

- O grafo acima possui 4 vértices e todos estão conectados entre si. Trata-se do grafo completo K_4 . Para validar a planaridade, testamos primeiro a Segunda Fórmula de Euler: $E \leq 3V - 6$. Temos $V = 4$ e $E = 6$. Logo, $6 \leq 3(4) - 6 \Rightarrow 6 \leq 6$ (condição válida). Além disso, como ele não contém o subgrafo proibido K_5 ou $K_{3,3}$, ele é indubitavelmente planar.
- Para eliminar o cruzamento central entre a aresta (A, C) e (B, D) , basta desenhar uma das diagonais pelo lado de fora:



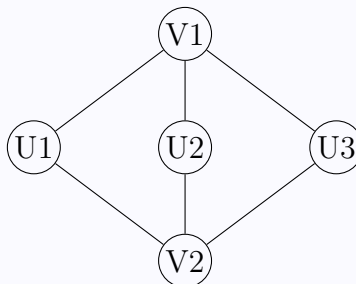
Nenhuma aresta se cruza, provando a planaridade.

8.4. **Prática de Fixação: Planaridade de Bipartidos ($K_{3,2}$).** Considere o grafo bipartido completo com partições de tamanho 3 e 2, desenhado abaixo com cruzamentos evidentes:

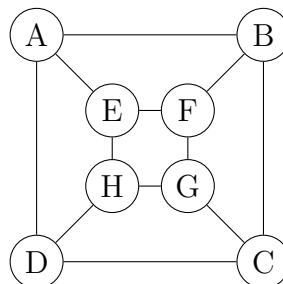


- Sabendo que o subgrafo proibido é o $K_{3,3}$, o grafo acima é planar? Justifique.
- Desenhe uma representação isomorfa do $K_{3,2}$ que não possua nenhum cruzamento de arestas.

- Sim, ele é planar. Primeiro, aplicamos a Segunda Fórmula de Euler ($E \leq 3V - 6$). Temos $V = 5$ e $E = 6$. Logo, $6 \leq 3(5) - 6 \Rightarrow 6 \leq 9$ (válido). Passando por esse filtro, analisamos por Kuratowski: como ele não contém um subgrafo $K_{3,3}$ (pois só tem 2 nós em uma partição) e nem K_5 , ele pode ser planificado sem cruzamentos.
- Para planificar, podemos colocar os nós da partição 2 (V1 e V2) um no "teto" e um no "chão", com a partição 3 alinhada no meio:



8.5. **Prática de Fixação: Fórmula de Euler em Cubos.** A figura abaixo mostra o grafo do Hexaedro (cubo), desenhado como dois quadrados concêntricos conectados.



- Determine o número de vértices (V) e de arestas (E) do grafo do cubo.
- Quantas **faces** (regiões planares) esse grafo possui? Liste-as intuitivamente e prove utilizando a Fórmula de Euler ($V - E + F = 2$). Lembre-se da face externa!



- a. O cubo tem $V = 8$ vértices e $E = 12$ arestas (4 do quadrado maior, 4 do menor, e 4 conectando-os).
- b. Há 6 faces no total ($F = 6$): 1 interna (EFGH), 4 faces trapezoidais nas laterais (ABFE, BCGF, CDHG, DAEH) e a **face externa** infinita (fora do quadrado ABCD). Provando pela Fórmula de Euler: $8 - 12 + 6 = 2 \Rightarrow 14 - 12 = 2 \Rightarrow 2 = 2$.

8.6. **Teoria: Segunda Fórmula de Euler e a Desigualdade de Planaridade.** Além de $V - E + F = 2$, grafos planares obedecem a uma relação baseada nos graus das faces (número de arestas que delimitam cada região). A soma dos graus de todas as faces é sempre o dobro do número de arestas: $\sum \text{graus das faces} = 2E$.

- a. Em um grafo simples (sem laços ou arestas múltiplas), o menor ciclo possível é um triângulo. Logo, o grau de qualquer face em um grafo simples não-árvore é no mínimo 3. Substitua essa restrição na segunda fórmula e combine o resultado com a primeira Fórmula de Euler para deduzir a desigualdade fundamental dos grafos planares: $E \leq 3V - 6$. Mostre o algebrismo passo a passo.
- b. Utilize essa nova desigualdade matemática ($E \leq 3V - 6$) para provar de forma absoluta e algébrica que o grafo completo K_5 ($V = 5, E = 10$) não tem como ser planar.

- a. Sabemos que $\sum \text{graus das faces} = 2E$. Como cada face tem no mínimo grau 3, a soma dos graus é no mínimo $3F$. Logo: $3F \leq 2E \Rightarrow F \leq \frac{2E}{3}$. Pela primeira Fórmula de Euler ($F = 2 - V + E$), substituímos F :

$$2 - V + E \leq \frac{2E}{3}$$

Multiplicando por 3:

$$6 - 3V + 3E \leq 2E$$

Passando $2E$ para a esquerda e o resto para a direita:

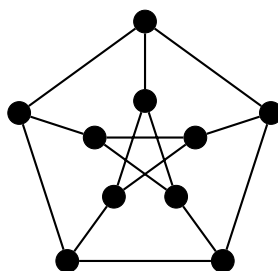
$$3E - 2E \leq 3V - 6 \Rightarrow E \leq 3V - 6$$

- b. Para o K_5 , temos $V = 5$ e $E = 10$. Se ele fosse planar, deveria satisfazer obrigatoriamente a desigualdade matemática deduzida:

$$10 \leq 3(5) - 6 \Rightarrow 10 \leq 15 - 6 \Rightarrow 10 \leq 9$$

Como $10 \leq 9$ é uma evidente **contradição matemática absurda**, provamos inquestionavelmente que o K_5 não pode ser planar!

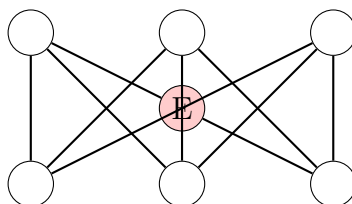
8.7. **Prática de Fixação: Teorema de Kuratowski.** O *Grafo de Petersen* é um famoso grafo regular de grau 3 desenhado com uma estrela inscrita em um pentágono, onde os vértices da estrela estão ligados aos do pentágono, cruzando várias arestas.



- a. É impossível redesenhar o Grafo de Petersen ($V = 10, E = 15$) no plano 2D sem cruzamentos. Verifique se ele passa no filtro inicial da Fórmula de Euler. Em seguida, identifique qual **Teorema Fundamental** embasa essa impossibilidade absoluta, mesmo ele passando no filtro geométrico.

- a. Para verificar a restrição de Euler ($E \leq 3V - 6$), substituímos os valores: $15 \leq 30 - 6 \Rightarrow 15 \leq 24$. A desigualdade é satisfeita perfeitamente, o que significa que ele **poderia** ser planar do ponto de vista da densidade de arestas. Contudo, a impossibilidade definitiva é governada pelo **Teorema de Kuratowski**. Ele dita que se o grafo é não planar, ele **obrigatoriamente** possui como subgrafo uma estrutura isomorfa a um **subgrafo de Kuratowski** (K_5 ou $K_{3,3}$) ou uma subdivisão deles. O Petersen oculta uma subdivisão estrutural do $K_{3,3}$.

- 8.8. **Prática de Fixação: Subdivisão de Kuratowski.** Abaixo temos o famoso Grafo das Utilidades ($K_{3,3}$) modificado: um nó extra (E) foi injetado bem no meio da aresta que conectava o nó superior esquerdo ao inferior direito.



- a. Pelas regras de Kuratowski, a quebra de uma aresta pela inserção de um nó de grau 2 (subdivisão) torna o grafo planar?
- b. É possível remover o nó E e reconectar a aresta livremente sem alterar o estado de planaridade do grafo original?

- a. **Não.** Primeiro, a Segunda Fórmula de Euler ($E \leq 3V - 6$) para esse grafo ($V = 7, E = 10$) nos dá $10 \leq 21 - 6 \Rightarrow 10 \leq 15$. O grafo passa nesse teste preliminar, parecendo planar matematicamente. Contudo, o Teorema de Kuratowski estabelece que qualquer grafo que contenha um $K_{3,3}$, um K_5 , ou uma subdivisão deles (que é exatamente o caso deste grafo com o nó E de grau 2 interrompendo a aresta) continua sendo intrinsecamente **não planar**.



- b. **Sim.** Para testar planaridade pela redução de Kuratowski, podemos "alisar" (remover) vértices de grau 2 conectando seus vizinhos diretamente. Ao remover **E** e ligar as pontas, o grafo volta a ser exatamente o $K_{3,3}$ original, provando sua não-planaridade.

9 Complexidade Computacional: P vs NP

1. **Introdução Prática à Complexidade.** Ao longo deste curso, estudamos diversos algoritmos de grafos. Alguns podem ser executados instantaneamente em grandes redes, enquanto outros podem demorar anos se o grafo crescer muito.
- a. Defina, com suas palavras e focando na prática de execução, o que é um algoritmo da classe **P** e um algoritmo da classe **NP**.
- b. Classifique os seguintes problemas estudados como P ou NP-Completo/NP-Difícil: (1) Encontrar o Menor Caminho (Dijkstra), (2) Ordenação Topológica (Kahn), (3) Coloração Mínima de Vértices, (4) Cobertura Mínima de Vértices.

- a. Algoritmos da classe **P** são aqueles que conseguimos **resolver** rapidamente (em tempo polinomial, como $O(V^2)$ ou $O(V + E)$). A classe **NP** engloba problemas cujas soluções propostas podem ser **verificadas** rapidamente (ex: é fácil checar se uma coloração dada está certa), mas para os quais não conhecemos nenhuma forma rápida (polinomial) de **encontrar** a solução ótima do zero (exigindo tempo exponencial $O(2^V)$ no pior caso).
- b. (1) Menor Caminho (Dijkstra) = **P**. (2) Ordenação Topológica (Kahn) = **P**. (3) Coloração Mínima = **NP**. (4) Cobertura Mínima = **NP**.

10 Coloração de Grafos

10.1. Número Cromático e Bipartição:

- a. O que significa "colorir vértices" de um grafo? O que é o **número cromático** $\chi(G)$?
- b. Se um grafo tem $\chi(G) = 2$, o que isso garante sobre a estrutura do grafo? E se o grafo contiver ciclos ímpares (ex: um triângulo)?
- c. Cite a aplicação do Teorema das Quatro Cores.

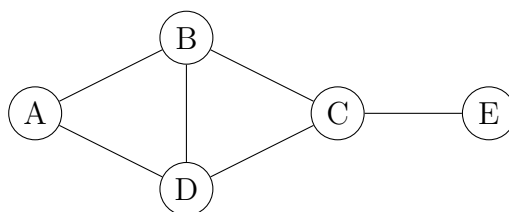
- a. Colorir os vértices significa atribuir rótulos (cores) a eles de modo que vértices adjacentes (conectados por uma aresta) nunca tenham a mesma cor. O número cromático $\chi(G)$ é o **número mínimo** de cores necessárias para colorir o grafo validamente.
- b. Se $\chi(G) = 2$, o grafo é **bipartido**. Se ele contiver qualquer ciclo de



tamanho ímpar (como um triângulo K_3), é matematicamente impossível colori-lo com apenas 2 cores (o terceiro vértice precisará de uma terceira cor para não conflitar com os outros dois), logo ele não será bipartido e terá $\chi(G) \geq 3$.

- c. O Teorema das Quatro Cores garante que qualquer grafo planar pode ser colorido com no máximo 4 cores. É usado classicamente na cartografia para provar que mapas geográficos podem ser coloridos sem que países fronteiriços tenham a mesma cor usando apenas 4 lápis.

10.2. **Prática: Algoritmo de Welsh-Powell.** Considere o grafo abaixo:



- a. Descreva os passos principais do Algoritmo Guloso de Welsh-Powell.
b. Aplique o algoritmo ao grafo acima preenchendo a tabela: liste os vértices ordenados por grau e a cor atribuída a cada um.
c. Qual é o número cromático $\chi(G)$ deste grafo?

- a. Passos do Welsh-Powell: (1) Calcular o grau de cada vértice; (2) Ordenar os vértices em ordem **decrecente** de grau; (3) Pegar a primeira cor não usada e colorir o primeiro vértice da lista que ainda não tem cor; (4) Percorrer a lista do início ao fim e colorir com essa mesma cor todos os vértices não coloridos que não sejam adjacentes a *nenhum* vértice que acabou de receber esta cor; (5) Repetir os passos 3 e 4 com uma nova cor até todos estarem coloridos.
b. Graus: B(3), C(3), D(3), A(2), E(1). Ordem (decrecente): B, C, D, A, E.

Vértice	Grau	Avaliação	Cor Atribuída
B	3	Primeiro sem cor.	Cor 1
C	3	Adjacente a B (já Cor 1). Pula.	-
D	3	Adjacente a B (já Cor 1). Pula.	-
A	2	Adjacente a B (já Cor 1). Pula.	-
E	1	Não adjacente a nenhum Cor 1 (só a C).	Cor 1

Nova iteração (próximo sem cor é C):



Pontifícia Universidade Católica de Minas Gerais

Inst. de Ciências Exatas e Informática, Campus Lourdes

Curso de Engenharia de Computação

Vértice	Grau	Avaliação	Cor Atribuída
C	3	Primeiro sem cor.	Cor 2
D	3	Adjacente a C (já Cor 2). Pula.	-
A	2	Não adjacente a C.	Cor 2

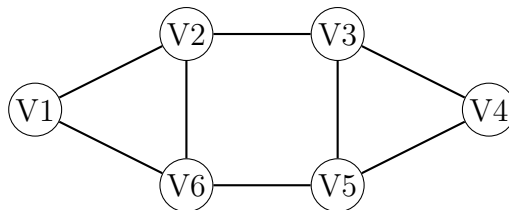
Nova iteração (próximo sem cor é D):

Vértice	Grau	Avaliação	Cor Atribuída
D	3	Primeiro sem cor.	Cor 3

Cores finais: B e E (Cor 1); C e A (Cor 2); D (Cor 3).

- c. O algoritmo precisou de 3 cores. Como o grafo contém um triângulo completo entre B, C e D (clique máxima $\omega(G) = 3$), sabemos que o número mínimo possível é 3. Logo, $\chi(G) = 3$.

10.3. **Prática de Fixação: Welsh-Powell (6 Vértices).** Considere o seguinte grafo:



- a. Construa a tabela de graus, ordene os vértices de forma decrescente e aplique o Welsh-Powell. Para desempate de graus iguais, use a ordem alfabética. Mostre a cor final de cada nó.

- a. Graus: V1(2), V2(3), V3(3), V4(2), V5(3), V6(3). Ordem (Decrescente + Alfabética): **V2, V3, V5, V6, V1, V4**.

Vértice	Grau	Ação (Conflitos verificados)	Cor
V2	3	Primeiro não colorido.	Cor 1
V3	3	Adjacente a V2 (Cor 1).	-
V5	3	Não adjacente a V2. Recebe Cor 1.	Cor 1
V6	3	Adjacente a V2 e V5.	-
V1	2	Adjacente a V2.	-
V4	2	Adjacente a V5 (Cor 1).	-

Nova rodada para os não coloridos (V3, V6, V1, V4):

Vértice	Grau	Ação (Conflitos verificados)	Cor
V3	3	Primeiro não colorido.	Cor 2
V6	3	Não adjacente a V3. Recebe Cor 2.	Cor 2
V1	2	Adjacente a V6 (Cor 2).	-
V4	2	Adjacente a V3 (Cor 2).	-

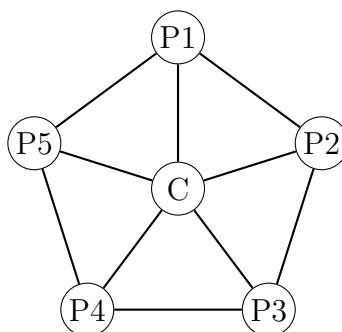


Nova rodada para (V1, V4):

Vértice	Grau	Ação (Conflitos verificados)	Cor
V1	2	Primeiro não colorido.	Cor 3
V4	2	Não adjacente a V1. Recebe Cor 3.	Cor 3

Cores finais: **Cor 1 (V2, V5), Cor 2 (V3, V6), Cor 3 (V1, V4).**
 $\chi(G) = 3$.

- 10.4. **Prática de Fixação: Welsh-Powell (Grafo Estrela).** Considere um grafo no formato de "Roda" ou "Estrela" desenhado com um nó central (C) conectado a 5 nós periféricos (P1 a P5) que formam um pentágono:



- Aplice o algoritmo de Welsh-Powell neste grafo *Roda* (W_6). Mostre os passos da coloração.
- Qual é a cor do nó central C? Quantas cores foram necessárias no total?

- a. Graus: C(5), P1(3), P2(3), P3(3), P4(3), P5(3). Ordem (Decrescente): **C, P1, P2, P3, P4, P5.**

Vértice	Grau	Ação (Conflitos verificados)	Cor
C	5	Primeiro não colorido.	Cor 1
P1	3	Adjacente a C (Cor 1).	-
P2	3	Adjacente a C (Cor 1).	-
P3	3	Adjacente a C (Cor 1).	-
P4	3	Adjacente a C (Cor 1).	-
P5	3	Adjacente a C (Cor 1).	-

Nova rodada (P1, P2, P3, P4, P5):

Vértice	Grau	Ação (Conflitos verificados)	Cor
P1	3	Primeiro não colorido.	Cor 2
P2	3	Adjacente a P1 (Cor 2).	-
P3	3	Não adjacente a P1. Recebe Cor 2.	Cor 2
P4	3	Adjacente a P3 (Cor 2).	-
P5	3	Adjacente a P1 (Cor 2).	-

Nova rodada (P2, P4, P5):



Vértice	Grau	Ação (Conflitos verificados)	Cor
P2	3	Primeiro não colorido.	Cor 3
P4	3	Não adjacente a P2. Recebe Cor 3.	Cor 3
P5	3	Adjacente a P4 (Cor 3).	-

Nova rodada (P5):

Vértice	Grau	Ação	Cor
P5	3	Primeiro não colorido.	Cor 4

- b. O nó central **C** fica com a **Cor 1** isolado, pois todos os outros nós conectam nele. No total, foram necessárias **4 cores** (Cor 1 para C, Cor 2 para P1/P3, Cor 3 para P2/P4, Cor 4 para P5). $\chi(W_6) = 4$.

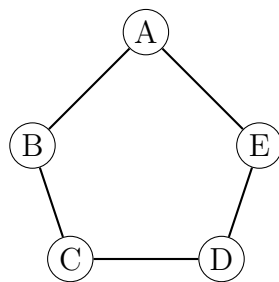
11 Coloração de Arestas e Algoritmo de Misra & Gries

2. **Teoria: Índice Cromático e Teorema de Vizing.** Enquanto a coloração de vértices evita que nós adjacentes tenham a mesma cor, a coloração de arestas impede que arestas incidentes ao mesmo nó compartilhem a cor. O número mínimo de cores de arestas é denotado pelo Índice Cromático $\chi'(G)$.

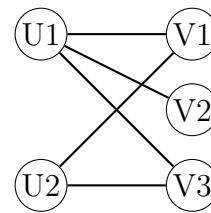
- a. O que determina o limite inferior do número cromático de arestas? Por que não podemos usar menos cores que esse valor?
- b. Enuncie o Teorema de Vizing e explique como ele simplifica drasticamente a complexidade do problema.

- a. O limite inferior natural é o grau máximo do grafo, denotado por $\Delta(G)$. Como um vértice com grau Δ tem Δ arestas conectadas a ele, cada uma dessas arestas precisa de uma cor diferente. Logo, $\chi'(G) \geq \Delta(G)$.
- b. O Teorema de Vizing afirma que, para qualquer grafo simples, o índice cromático $\chi'(G)$ é estritamente igual a $\Delta(G)$ ou $\Delta(G) + 1$. Ou seja, o número exato de cores só pode assumir um desses dois valores (Classe 1 ou Classe 2).

3. **Prática: Coloração de Arestas em Ciclos e Bipartidos.** Sabemos que o Índice Cromático $\chi'(G)$ de um grafo bipartido é sempre exatamente igual a Δ (Teorema de König), enquanto ciclos ímpares exigem $\Delta + 1$.



Grafo C_5

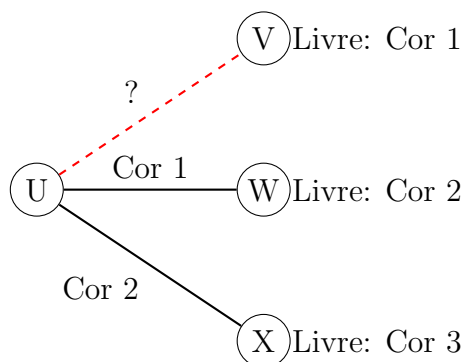


Grafo Bipartido

- Qual é o grau máximo Δ do grafo C_5 ? Tente colorir as arestas de C_5 usando apenas Δ cores. Por que falha e obriga o uso de $\Delta + 1$ cores?
- Qual é o grau máximo Δ do grafo Bipartido? Pelo Teorema de König, qual é o número cromático de arestas $\chi'(G)$ desse grafo? Atribua cores válidas às arestas provando a afirmação.

- O ciclo C_5 tem $\Delta = 2$. Se tentarmos colorir com 2 cores (ex: Vermelho e Azul): (A,B) Vermelho, (B,C) Azul, (C,D) Vermelho, (D,E) Azul. Agora falta a aresta (E,A). O nó E já tem uma aresta Azul, então (E,A) precisaria ser Vermelho. Mas o nó A já tem uma aresta Vermelha! O conflito é inevitável em ciclos ímpares. Precisamos de uma 3ª cor. Logo, $\chi'(C_5) = 3$ (que é $\Delta + 1$, Classe 2).
- O grafo Bipartido tem $\Delta = 3$ (o nó U1 se conecta a 3 nós). O Teorema de König garante que grafos bipartidos sempre são Classe 1, ou seja, $\chi'(G) = \Delta = 3$. Exemplo de coloração: (U1,V1) Cor 1, (U1,V2) Cor 2, (U1,V3) Cor 3. Para o nó U2: (U2,V1) Cor 2, (U2,V3) Cor 1. Nenhuma cor entra em conflito e usamos exatamente 3 cores.

4. **Prática: Misra & Gries (Rotação de Leque Simples).** O algoritmo de Misra & Gries colore as arestas iterativamente usando a "Dança das Cadeiras" em um Leque (Fan) de arestas.



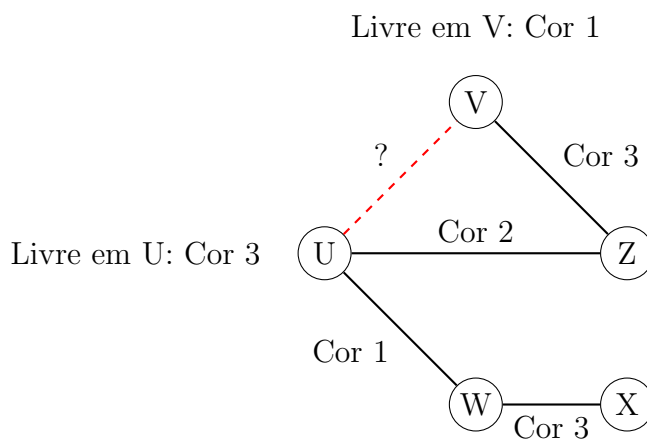
Desejamos colorir a aresta livre (U, V) . As cores do sistema são $\{Cor1, Cor2, Cor3\}$. O nó U já usa as cores 1 e 2.



- Identifique as cores livres do nó U e as do nó V.
- Construa o Leque (Fan) a partir de V em relação ao nó U. Liste a sequência de nós $F_0, F_1, \dots$
- Gire as cores do Leque. Qual cor a aresta (U, V) receberá no final do giro?

- U usa as cores 1 e 2, então a cor livre de U é a **Cor 3**. A cor livre de V, indicada no diagrama, é a **Cor 1**.
- O leque começa em $F_0 = V$. A cor livre de V é **Cor 1**. Procuramos quem usa a Cor 1 conectada a U: é o nó W. Então $F_1 = W$. A cor livre de W é **Cor 2**. Procuramos quem usa a Cor 2 em U: é o nó X. Então $F_2 = X$. A cor livre de X é a Cor 3, que é exatamente a cor livre de U! O Leque termina. Sequência: **V, W, X**.
- Rotacionamos o leque empurrando as cores "para baixo". A aresta (U, W) perde a Cor 1 para a aresta (U, V). A aresta (U, X) perde a Cor 2 para a aresta (U, W). A última aresta do leque (U, X) fica com a cor livre de U (**Cor 3**). Assim, a aresta (U, V) recebe a **Cor 1**.

5. **Prática: Misra & Gries (O Impasse e a Cadeia de Kempe).** Tentar montar um Leque nem sempre dá certo. Veja o cenário abaixo, onde queremos colorir (U, V) com as cores $\{1, 2, 3\}$.



Note o estado do grafo: - U usa as cores 1 e 2. Livre de U = **Cor 3**. - V usa a cor 3. Livre de V = **Cor 1** (e 2). Vamos focar na Cor 1. - W usa as cores 1 e 3. Livre de W = **Cor 2**. - Z usa as cores 2 e 3. Livre de Z = **Cor 1**.

- Tente construir o Leque a partir de V (focando na cor livre 1). Mostre passo a passo por que o Leque entra em um "loop" e trava sem encontrar a cor livre de U (Cor 3).
- Como o leque falhou, precisamos usar uma Cadeia de Kempe. As cores do impasse são a Livre de U (Cor 3) e a Livre de V (Cor 1). Partindo do nó V, rastreie o caminho alternante (1-3). Quais nós compõem essa cadeia?

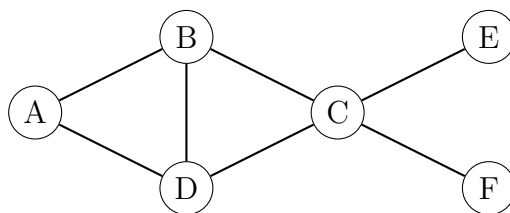


- c. Inverta as cores da cadeia que você encontrou. O que acontece com a cor livre do nó V ? Como isso resolve o problema e permite colorir a aresta (U, V) imediatamente?

- a. Construção do Leque: Começamos em $F_0 = V$ (Livre = 1). Olhamos quem se conecta a U com Cor 1: é o nó W . Então $F_1 = W$. A cor livre de W é a Cor 2. Olhamos quem se conecta a U com Cor 2: é o nó Z . Então $F_2 = Z$. A cor livre de Z é a Cor 1. Olhamos quem se conecta a U com a Cor 1: é o nó W de novo! O leque fez um loop ($V \rightarrow W \rightarrow Z \rightarrow W$) e **travou**. Não conseguimos achar nenhum nó no leque que tenha a Cor 3 livre (que é a cor que U precisa para fechar a rotação).
- b. Cadeia de Kempe $(1, 3)$: Partindo de V , procuramos arestas que alternam entre Cor 1 e Cor 3. V está conectado a Z usando a **Cor 3**. Agora, a partir de Z , procuramos uma aresta com a Cor 1... mas Z não tem nenhuma aresta com Cor 1 (a livre dele é 1). O caminho acaba aí! A cadeia inteira é apenas o trecho **$V - Z$** .
- c. Invertendo as cores da cadeia $V-Z$, a aresta que era Cor 3 passa a ser **Cor 1**. Agora veja a mágica: o nó V trocou sua aresta 3 pela 1, o que significa que a **Cor 3 ficou livre em V** ! Como o nó U já tinha a Cor 3 livre desde o começo, o conflito desapareceu. Agora basta pintar a aresta tracejada (U, V) com a **Cor 3** e o algoritmo termina com sucesso.

12 Independência e Cobertura de Vértices

- 12.1. **Prática: Algoritmo Guloso para Cobertura de Vértices.** O problema de Cobertura de Vértices Mínima é NP-Completo, mas uma **aproximação gulosa** garante uma cobertura no máximo duas vezes pior que a ótima. Considere o grafo a seguir:



- a. Execute a Aproximação Gulosa (escolhendo iterativamente a aresta de maior impacto heurístico ou aleatória, adicionar ambos os nós à cobertura, e remover todas as arestas incidentes a eles). Mostre a tabela de rastreo escolhendo a aresta (C, B) no primeiro passo.

- a. **Rastreo do Guloso 2-Aprox:** O algoritmo seleciona uma aresta ainda existente, adiciona seus 2 extremos na Cobertura (Vertex Cover) e remove

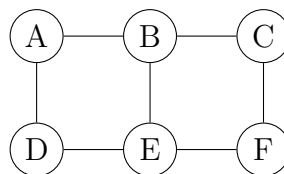


do grafo todas as arestas que tocam neles.

Passo	Aresta Escolhida	Vértices Adicionados à Cobertura K	Arestas Removidas (Cobertas)
1	(C, B)	C, B	(C, B) , (A, B) , (D, B) , (C, D) , (C, E) , (C, F)
2	(A, D)	A, D	(A, D) (era a única aresta sobrevivente!)

O conjunto final de cobertura gerado pela heurística é $K = \{B, C, A, D\}$. (Note que E e F ficaram de fora, mas suas arestas já haviam sido cobertas por C).

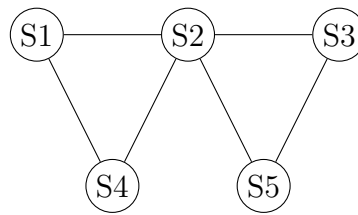
12.2. **Prática de Fixação: Teorema de Gallai em Grades.** Considere o grafo em formato de grade 3x2 abaixo.



- Pinte os vértices para encontrar o **Conjunto Independente Máximo** (IS_{max}). Quais são esses vértices e qual o tamanho da independência máxima $\alpha(G)$?
- Usando a relação de complementaridade do Teorema de Gallai ($|IS| + |VC| = |V|$), calcule e identifique quais vértices formam a **Cobertura de Vértices Mínima** (VC_{min}).

- Um conjunto independente é formado por vértices que não se tocam. Para maximizar, escolhemos alternadamente: A, C e E (nenhum deles tem aresta com outro do grupo). O tamanho é $\alpha(G) = 3$. (Outro válido: B, D, F).
- Se o grafo tem $|V| = 6$ vértices e o Conjunto Independente ocupa 3, o Teorema de Gallai garante que os $6 - 3 = 3$ vértices restantes formam a Cobertura de Vértices Mínima. Portanto, a VC_{min} é formada pelo complementar: **$\{B, D, F\}$** . Qualquer aresta do grafo toca em pelo menos um desses nós.

12.3. **Prática de Fixação: Conjunto Dominante (Roteadores Wi-Fi).** Você precisa instalar o número mínimo de roteadores em um escritório. Cada roteador ilumina a sala onde está e as salas vizinhas adjacentes. Isso equivale a achar um **Minimum Dominating Set (MDS)**.



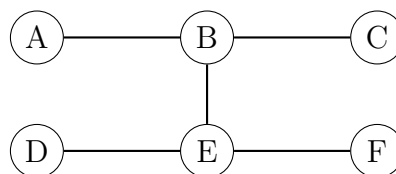
- Um conjunto independente é automaticamente um conjunto dominante? Dê um exemplo no grafo acima.
- Qual é o Conjunto Dominante Mínimo para este escritório? Quantos roteadores são necessários e onde devem ser posicionados?

- Nem sempre. Um Conjunto Independente Maximal é sempre dominante, mas um qualquer não. Por exemplo, $\{S1, S3\}$ é independente, mas **não** domina o nó $S5$. Logo, falha como dominante.
- Para dominar todos os 5 nós com o mínimo de peças, focamos em nós de alto grau ($S2$). Colocando um roteador em **$S2$** , ele domina a si mesmo e ilumina $S1, S3, S4$ e $S5$ de uma só vez. Apenas **1 roteador** no nó **$S2$** é o Conjunto Dominante Mínimo.

12.4. **Prática de Fixação: Algoritmo Guloso para Conjunto Dominante.** Uma heurística clássica para encontrar um Conjunto Dominante (D) é a seguinte:

- Passo 1:** Escolha o vértice que cobre (domina) o maior número de vértices **ainda não cobertos** (incluindo a si mesmo). Em caso de empate, escolha pela ordem alfabética.
- Passo 2:** Repita até que todos os vértices do grafo estejam cobertos.

Aplique essa heurística no grafo abaixo:



- Demonstre o rastreamento passo a passo da heurística, indicando qual vértice foi escolhido em cada iteração e quais vértices ele passou a cobrir.
- Qual foi o Conjunto Dominante final gerado pela heurística? Ele é ótimo?

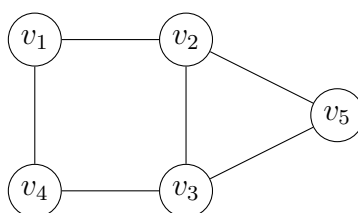
- Iteração 1:** Inicialmente, nenhum vértice está coberto. Coberturas possíveis: $A(2), B(4), C(2), D(2), E(4), F(2)$. Maior cobertura: B e E (ambos com 4). Pelo desempate alfabético, escolhemos **B** . Vértices cobertos agora: $\{A, B, C, E\}$. Vértices restantes não cobertos: $\{D, F\}$.



Iteração 2: Recalculamos a cobertura apenas para os nós **ainda não cobertos** $\{D, F\}$. - D cobre a si mesmo: $\{D\}$ (1 nó). - E cobre $\{D, F\}$ (2 nós). - F cobre a si mesmo: $\{F\}$ (1 nó). Maior cobertura: **E**. Escolhemos E. Vértices cobertos agora: todos. Fim do algoritmo.

- b. O Conjunto Dominante gerado foi $D = \{B, E\}$. Sim, ele é ótimo, pois é impossível dominar todos esses 6 nós (especialmente as 4 pontas A, C, D, F que não se conectam entre si) usando apenas 1 vértice.

12.5. **Prática: Identificação de IS e VC.** Considere o grafo simples abaixo:



- a. Encontre uma **Cobertura de Vértices Mínima** (VC_{min}) para este grafo. Quais vértices fazem parte?
- b. Encontre o **Conjunto Independente Máximo** (IS_{max}) usando a propriedade complementar.
- c. Demonstre que a equação $|IS_{max}| + |VC_{min}| = |V|$ é satisfeita neste grafo.

- a. Para cobrir todas as arestas com o mínimo de vértices, precisamos priorizar vértices de alto grau. Escolhendo v_2 e v_3 , cobrimos as arestas: (v_1, v_2) , (v_2, v_3) , (v_2, v_5) , (v_3, v_4) , (v_3, v_5) . A única aresta que falta cobrir é (v_1, v_4) . Para cobri-la, podemos escolher v_1 (ou v_4). Logo, $VC_{min} = \{v_1, v_2, v_3\}$ (também seria válido $\{v_4, v_2, v_3\}$).
- b. O conjunto independente máximo é o complemento da cobertura mínima: $IS_{max} = V - VC_{min} = \{v_1, v_2, v_3, v_4, v_5\} - \{v_1, v_2, v_3\} = \{v_4, v_5\}$. Podemos verificar visualmente que v_4 e v_5 não possuem nenhuma aresta conectando-os e não há como adicionar um 3º vértice sem criar adjacência (pois v_1, v_2, v_3 são adjacentes a pelo menos um deles).
- c. O grafo possui 5 vértices ($|V| = 5$). O tamanho do Conjunto Independente Máximo é 2 ($|\{v_4, v_5\}| = 2$). O tamanho da Cobertura de Vértices Mínima é 3 ($|\{v_1, v_2, v_3\}| = 3$). A equação é: $2 + 3 = 5$, o que comprova a relação estrutural pelo Teorema de Gallai.